

Production-Grade Home Decor E-Commerce Development Plan

Project Type: Premium Luxury Home Decor E-Commerce Store **Stack:** Next.js 16 (App Router), React 19, Tailwind v4, Supabase (PostgreSQL), Prisma, Motion, GSAP, Lenis **Target Vibe:** Refined, high-end, luxury (Zara Home, West Elm aesthetic) **Note to AI Developers:** This is a **PRODUCTION-GRADE** project, not a prototype. Code must be highly modular, typed (TypeScript), scalable, and utilize a strict global design system and CSS variables. Do not take shortcuts.

Phase 1: Foundation & Global Architecture Setup (Weeks 1-2)

Goal: Establish the strict global design system, typography, color palettes, and core repository architecture. The foundation must be flawless before any components are built.

- **1.1 Repository & Project Initialization:**

- Initialize Next.js 16 app with TypeScript, ESLint, and Prettier.
- Configure `tsconfig.json` with strict type checking and custom path aliases (e.g., `@/components`, `@/lib`).

- **1.2 Global Design System & Tailwind v4 Configuration:**

- **Crucial AI Instruction:** Define all foundational tokens (colors, spacing, typography) as CSS variables in `globals.css` and map them in `tailwind.config.js`.
- Establish font stacks (e.g., primary serif for headings, clean sans-serif for body).
- Implement fluid typography utilizing CSS `clamp()` and container queries.

- **1.3 Premium UI Component Library Integration:**

- Install and configure `shadcn/ui` with Radix UI primitives.
- Customize `shadcn` components immediately to match the luxury aesthetic (remove heavy borders, refine radii, apply soft shadows).
- Integrate Lucide React for consistent iconography.

- **1.4 Animation Engine & Smooth Scroll Setup:**

- Integrate **Lenis** globally for inertia-based smooth scrolling.
- Set up the global provider for **Motion** (Framer Motion) and configure default spring physics for luxury easing.

- Install **GSAP** and `ScrollTrigger` for future advanced timelines.

- **1.5 Database & ORM Initialization (Supabase + Prisma):**

- Set up Supabase project.
- Initialize Prisma and connect to the Supabase PostgreSQL instance.
- Design the core schema models: User, Product, Category, Order, OrderItem.

Phase 2: Core Backend Services & Authentication (Weeks 3-4)

Goal: Build the robust, secure engine that will power the application.

- **2.1 Authentication System (Better Auth):**

- Implement user sign-up/login using email/password and OAuth (Google).
- Set up secure session management and middleware to protect routes.

- **2.2 Database Schema Finalization & Migrations:**

- Expand Prisma schema to include Variants, Reviews, Cart, and Wishlist.
- Execute migrations and seed the database with mock luxury decor data.

- **2.3 API Route Handlers & Server Actions:**

- Create secure Next.js Server Actions for data mutations (e.g., `addToCart`, `createOrder`, `updateProfile`).
- Implement Zod validation for all incoming data payloads.

- **2.4 File Storage Integration:**

- Configure Supabase Storage buckets for product images and user avatars.
- Write utility functions for secure image upload and optimization.

Phase 3: Frontend Product Catalog & Luxury UI Construction (Weeks 5-6)

Goal: Translate the design system into a stunning, performant browsing experience.

- **3.1 Global Layouts & Navigation:**

- Build a responsive, glassmorphic header with a mega-menu and blur-on-scroll effects.
- Implement a sophisticated, staggered-link footer with subtle parallax.

- **3.2 Product Listing Page (PLP) & Search:**

- Build the product grid using CSS Grid and Subgrid.

- Implement **Meilisearch** for instant, typo-tolerant search and filtering (by category, price, material).
- **Animation Note:** Add subtle entrance animations and elegant hover states to product cards (soft lift, image zoom).
- **3.3 Product Detail Page (PDP):**
 - Create a high-fidelity image gallery with smooth transitions and zoom capabilities.
 - Build the product information section with elegant typography, variant selectors, and accordion details (materials, dimensions, care).
 - **Animation Note:** Use staggered text reveals and smooth "Add to Cart" micro-interactions.

Phase 4: E-Commerce Workflows & Checkout (Weeks 7-8)

Goal: Create a frictionless, trust-inspiring path to purchase.

- **4.1 Advanced Shopping Cart:**
 - Implement a slide-out cart drawer using Motion for smooth entry/exit.
 - Use Zustand for robust client-side cart state management.
 - Add micro-interactions for quantity adjustments and cart badge updates.
- **4.2 Multi-Step Checkout Flow:**
 - Design a clean, distraction-free checkout page.
 - Build forms utilizing React Hook Form + Zod for robust client-side validation.
- **4.3 Payment Gateway Integration:**
 - Integrate Stripe for international credit card processing.
 - Integrate localized Pakistani payment methods (JazzCash/Easypaisa API).
 - Implement secure webhook handlers to process successful transactions.
- **4.4 Transactional Emails:**
 - Set up Resend to send beautifully designed order confirmations and welcome emails.

Phase 5: Polish, Security & Production Readiness (Weeks 9-10)

Goal: Elevate the application from functional to exceptional, ensuring performance and security.

- **5.1 Advanced Animations & Micro-Interactions (The "Luxury Polish"):**
 - Implement SplitType for elegant hero section reveals.
 - Add GSAP ScrollTrigger for complex storytelling sections (e.g., brand history, artisan focus).
 - Introduce Lottie animations for empty states and success screens.

- **5.2 Security Hardening:**

- Implement Arcjet for bot protection and rate limiting.
- Audit Row Level Security (RLS) policies in Supabase.

- **5.3 Performance Optimization & Analytics:**

- Optimize Next.js image loading and font delivery.
- Integrate GA4, Meta Pixel, and Microsoft Clarity for user behavior tracking.

- **5.4 Final Testing & Deployment:**

- Conduct thorough QA (cross-browser, mobile responsiveness, payment flow testing).
- Deploy frontend to Vercel and configure the custom domain via Cloudflare.

Directives for AI Coding Assistants

When generating code for this project, adhere strictly to the following:

1. **Never output prototype code.** Assume every component is destined for production.
2. **Design System First:** Always use the predefined CSS variables and Tailwind configuration for colors, spacing, and typography. Do not hardcode arbitrary values.
3. **Refined Animations Only:** Adhere to the "premium luxury" animation guidelines. Avoid flashy, chaotic, or overly bouncy animations. Use subtle easing curves.
4. **Type Safety is Non-Negotiable:** Ensure all Next.js components, Server Actions, and API routes have comprehensive TypeScript interfaces.
5. **Modular Architecture:** Break down complex UI elements into small, reusable client or server components.